Database Systems Implementation
Mini-Project

1075201

Computer Science (Part C)

Hilary 2026

1

a

i

We manage tuple-level locks using shared/exclusive locks (XSLocks in Figure 1). These are standard two-phase locks: they operate in shared mode for queries and exclusive mode for edits (add/delete).

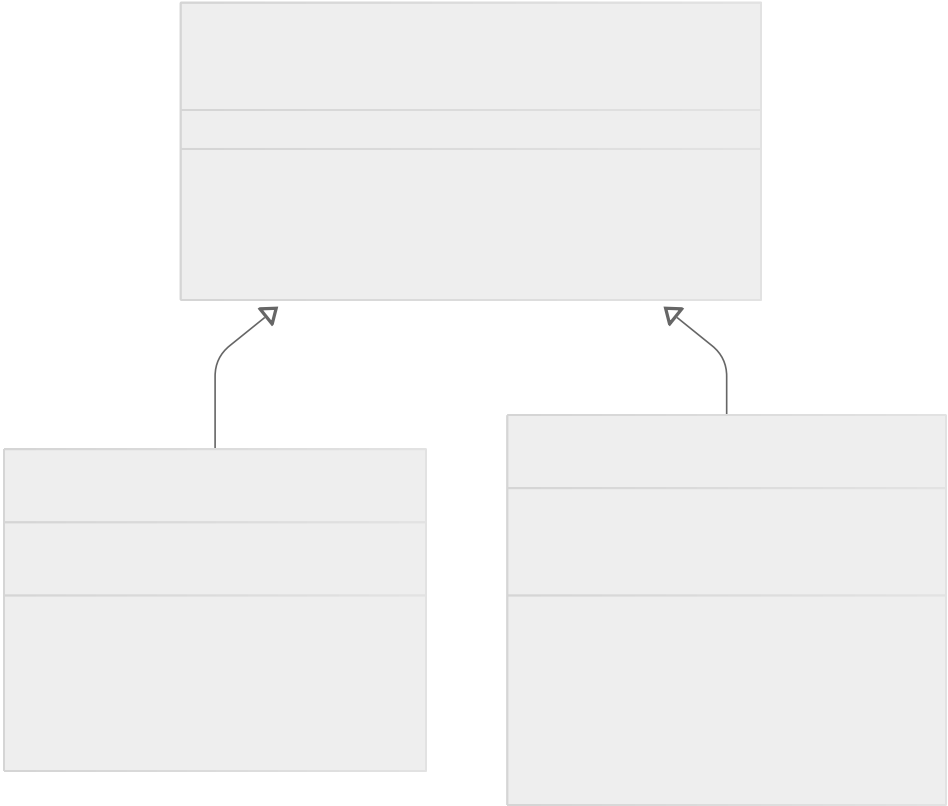

Figure 1: Lock Hierarchy

XSLocks are directly incorporated into the `DataTuple` class (Figure 2). The lock maintains an explicit set of `lock_holders` (`std::flat_set`) to support the deadlock detector's traversal of the waits-for graph. We use a flat set because the number of concurrent holders for a single tuple is typically very small, optimizing for cache locality.

Figure 2: DataTuple Class

Within each Lock, we maintain an explicit set of transactions holding the lock. This is later used by the deadlock detector to determine which transactions are waiting on which locks while traversing the implicit waits-for graph. We explicitly use inheritance and a VTable to implement the lock hierarchy,

rather than a C++20 concept. This allows each transaction to hold sets of Lock pointers and treat them uniformly.

ii

To support serializable isolation and prevent phantoms, we implement predicate locks using shared locks (SLocks in Figure 1). These locks represent conditions of the form "all tuples matching predicate P."

Figure 3: Relation and Group Classes

We provide three levels of granularity for predicate locks:

1. Relation Lock: Covers all tuples in a relation $R(x, y)$.
2. Group Locks: Cover subsets of a relation based on indices. These handle conditions of the form $R(c, x)$, $R(x, c)$, and $R(x, x)$ (diagonal).
3. Tuple Locks: These act as the finest grain of locking, locking individual tuples.

Relation locks belong directly to Relation objects (see Figure 3), while group locks are stored in groups belonging to relations (see Figure 3). The group locks are organized into three separate groups: one for the left attribute, one for the right attribute, and one for the diagonal (where left == right). This allows us to efficiently determine which locks to acquire based on the query predicates. We will later see in 1(c) how these granularities of locks neatly cover the types of queries we have to evaluate.

The different operations interact with the locks as follows:

Queries read ranges of tuples matching a predicate. For example, a query $R(1, Y)$ requires reading all tuples where the left attribute is 1 in relation R. To prevent phantoms, we acquire a shared lock on the relevant group lock (e.g., the group lock for left attribute 1). This ensures that if another transaction tries to insert a tuple that matches this predicate, it is blocked until the query transaction commits or rolls back. While reading existing tuples, we also need to prevent dirty reads from transactions that have not yet committed (or have aborted). Thus, queries use the `XSLock::permits_read` method, which checks whether the lock could be acquired in shared mode by this transaction.

There is an edge case for queries of the form $R(c_1, c_2)$, i.e. constant queries. For these queries, we do not take any predicate lock covering the (c_1, c_2) tuple in R, and thus must acquire the tuple-level lock rather than only using `XSLock::permits_read`.

Edit operations modify tuples from an explicit list. As they do not use implicit ranges, they do not need to acquire predicate locks directly. However, they still need to avoid causing phantoms, so for every tuple they edit, they must ensure that no other transaction has a shared lock on the relevant group lock covering that tuple's predicate. This is achieved using the `SLock::permits_edit` method, which checks whether the lock could be acquired in exclusive mode by this transaction. When editing a tuple, a transaction also acquires an exclusive lock on the tuple itself, as in standard two-phase locking.

Apart from the lock objects and their owners (Group, Relation and DataTuple), we also describe the Transaction objects (Figure 4) which hold onto pointers to locks.

Figure 4: Transaction Class

We track the locks held by each transaction in order to release the locks when the transaction commits or rolls back, and also track the locks that a transaction is relying on to make progress in order to detect deadlocks. A transaction likely holds many locks, so we use a hash set (std::unordered_set) to store the locks held by each transaction. On the other hand, we expect the number of locks that a transaction is waiting on to be small, so we use a std::flat_set to store the locks that a transaction is waiting on.

These transactions are stored in a single Database object (Figure 5) that acts as the global transaction and lock manager. When transactions finish an operation, they are either suspended or not. The Database class is responsible for detecting deadlocks in the former case and progressing normal control flow in the latter case. This is implemented in `Database::OnControl`.

Figure 5: Database Class

We move some of the deadlock detection logic into a separate `DeadlockDetector` class (Figure 6), composed into the Database class. This keeps the core transaction/locking management logic separate from deadlock detection.

Figure 6: Deadlock Detector Class

iii

The locks expose the following primary functions:

`Lock::acquire(tx_id: TID, mode: LockMode) -> boolean`

Attempts to register the transaction as a holder. For XSLocks, this includes checking for shared/exclusive compatibility (see Algorithm 1). If a conflict occurs (e.g., requesting EXCLUSIVE when another transaction holds the lock), it returns `false`. For SLocks, we can always add the transaction as a holder and return `true`.

`Transaction::acquire(lock: Lock, mode: LockMode) -> boolean`

A wrapper around `Lock::acquire` used by the execution engine. If `Lock::acquire` fails, the transaction is added to the lock's internal wait-queue and the transaction's `required_locks` set. The function returns `false` to signal that the transaction must be suspended.

`Lock::release(tx_id: TID)`
Removes the transaction from the holder set. This is called during commit or rollback via `Transaction::release_all_locks()` that not only releases the locks but also clears the transaction's `held_locks` set.

`Lock::current_holders() -> &Set<TID>`
Returns the transactions currently holding the lock, used by the deadlock detector to construct the waits-for graph.

Algorithm 1: XSLock::Acquire

```
procedure XSLock::Acquire(tx_id: TID, mode: LockMode) → boolean
    if mode = EXCLUSIVE then
        if holder_count() >= 2 then
            return false
        else if holder_count() = 1 and ¬ is_held_by(tx_id) then
            return false
        else
            # Upgrade or Acquire
            lock_holders.insert(tx_id)
            held_exclusively ← true
            return true
        end if
    else
        if held_exclusively then
            return is_held_by(tx_id)
        else
            lock_holders.insert(tx_id)
            return true
        end if
    end if
end procedure
```

The deadlock detector object provides the following function:

`DeadlockDetector::detect_cycle(tid: TID) -> Optional<TID>`
Performs a depth-first search starting from the given transaction ID, traversing through the waits-for graph by looking at the locks that the transaction is waiting on, and then looking at the transactions that are holding those locks, and so on. If we run into any back edge, then we have found a cycle. We then locate the youngest transaction in the cycle and return it as the victim to be rolled back. This is described in more detail in Algorithm 2.

Algorithm 2: Deadlock Detection

```
procedure DeadlockDetector::DetectCycle(start_tid: TID) → Optional<TID>
    visited_tx.clear()
    cycle.clear()
    if DFS(start_tid) then
```

```
    # Locate the youngest transaction in the cycle to abort
    victim ← cycle[0]
    for each tid in cycle do
        if transactions[tid].born_at > transactions[victim].born_at then
            victim ← tid
        end if
    end for
    return victim
  end if
  return empty
end procedure
procedure DeadlockDetector::DFS(tid: TID) → boolean
  if visited_tx[tid] = VISITING then return true
  if visited_tx[tid] = VISITED then return false
  visited_tx[tid] ← VISITING
  for each lock in transactions[tid].required_locks do
    for each holding_tid in lock.current_holders() do
      if holding_tid = tid then continue
      if DFS(holding_tid) then
        cycle.push(tid)
        return true
      end if
    end for
  end for
  visited_tx[tid] ← VISITED
  return false
end procedure
```

b

i

The tuples themselves are stored in the Relation class's tuples vector. We use a StableVector that works as a linked list of fixed-size blocks. This allows us to have stable pointers to tuples that can be stored in the various indices. Using a normal std::vector would require us to update all the pointers in the indices every time we resize the vector, while using a linked list would result in worse cache performance and allocation overhead. Note that we only ever add tuples to the end of the StableVector, and never remove tuples from the middle, deletion works by setting the alive flag to false in the DataTuple.

Based on the query patterns we will see later in 1(c), we maintain 3 kinds of hash indices on the relation. The first two are for queries of the form $R(c, Y)$ and $R(X, c)$ where we want to efficiently iterate over all tuples matching a constant on the left or right attribute respectively. The third index is for queries of the form $R(X, X)$ where we want to efficiently iterate over all tuples where the left and right attributes are the same. Since each of these indices cover the same range as the predicate locks described above, we also put them in the group class.

Note that we do not actually have a whole relation index. We opted not to do this because it would add more overhead to adding tuples while not providing a significant speedup since we can use our left/right indices to efficiently check if a given tuple already exists in the relation. To iterate over all tuples in the relation, we can directly iterate over the StableVector of tuples, which would be more efficient than using another index.

ii

We first discuss the methods exposed by the Relation class:

`Relation::get_tuple(left: uint32_t, right: uint32_t) -> DataTuple*`
retrieves a pointer to the DataTuple with the given left and right attributes. It creates the tuple if necessary, so it never returns null. Note that it does not guarantee that the tuple is alive, so the caller should check the alive flag if they only want alive tuples. This works by using `l_to_r_index` to find the group of tuples with the given left attribute, and then using `Group::find` to find the tuple with the given right attribute within that group.

`Relation::edit_tuple(left: uint32_t, right: uint32_t) -> boolean`
is used to add or delete a tuple from the relation. It first finds the DataTuple via `Relation::get_tuple`, then checks the relevant locks to prevent phantoms and tries to acquire an exclusive lock on the tuple. If any lock check or acquisition fails, it records the locks the transaction needs and returns false to signal that the transaction should be suspended. Otherwise, it updates the tuple's alive status and returns true.

Then we discuss the methods exposed by the Group class:

`Group::insert(tp: *DataTuple): unit` inserts the given tuple pointer into the group index. It is used when adding a new tuple to the relation and updating the indices.

`Group::find(left: uint32_t, right: uint32_t) -> DataTuple*` finds the tuple with the given left and right attributes in the group index. It returns null if no such tuple exists. It is used to efficiently locate tuples matching a given predicate when evaluating queries.

c

d & e

We use a single algorithm for both adding and deleting, with only a difference in the `new_alive` parameter. An important design consideration is that when a transaction fails either a lock permission check or a lock acquisition, all relevant locks are recorded in the transaction's `required_locks`. This helps ensure deadlocks are detected as early as possible (without needing to resume transactions that cannot make meaningful progress).

We also store the original tuple values to allow rollbacks.

Algorithm 3: Single Tuple Edit Algorithm

```
procedure Relation::edit_tuple(tx: &Transaction, left: uint32_t, right: uint32_t,
            new_alive: bool) → StatusCode
    tp ← get_tuple(left, right) # creates the tuple if it doesn't exist
    if permitted_by_locks(tx.tid, left, right) and tx.acquire(tp->lock, EXCLUSIVE) then
        if tp->alive ≠ new_alive then
            tx.store_original(tp)
            tx.num_modified++
            tp->alive ← new_alive
```

```
        endif
        return true;
    else
        insert_required_locks_for_edit(tx, left, right)
        return false;
    end if
end procedure
procedure Relation::permitted_by_locks(tx_id: TID, left: uint32_t, right: uint32_t)
            → boolean
    if ¬ whole_rel_lock.permits_edit(tx_id) then return false
    if left = right and ¬ diagonal_index.lock.permits(tx_id) then return false
    if ¬ l_to_r_index[left].lock.permits(tx_id) then return false
    if ¬ r_to_l_index[right].lock.permits(tx_id) then return false
    return true
end procedure
procedure Relation::insert_required_locks_for_edit(tx: &Transaction, left: uint32_t,
right: uint32_t) → unit
    # tp.lock already inserted via tx.acquire()
    tx.required_locks.insert(&whole_rel_lock)
    if left = right then
        tx.required_locks.insert(&diagonal_index.lock)
    end if
    tx.required_locks.insert(&l_to_r_index[left].lock)
    tx.required_locks.insert(&r_to_l_index[right].lock)
end procedure
```

f

When we commit, we just need to ensure that we are in a "ready" state to start a new operation and release all locks.

Algorithm 4: Commit Algorithm

```
procedure Transaction::commit() → StatusCode
    if state ≠ READY then
        print("Error: Trying to commit a transaction that is not ready.")
        return StatusCode::SUCCESS;
    end if
    for lock in held_locks do
        lock.release(tid)
    end for
    held_locks.clear()
    return StatusCode::SUCCESS;
end procedure
```

g

To rollback, we just need to restore the original values of the tuples and release the locks held.

Algorithm 5: Rollback Algorithm

```
procedure Transaction::rollback() → StatusCode
    for (tp, original_alive) in original_values do
        tp->alive ← original_alive
    end for
    for lock in held_locks do
        lock.release(tid)
    end for
    held_locks.clear()
    return StatusCode::SUCCESS;
end procedure
```

3

a

	Action	Time (ms)	# of Answers
Step 1	Import:	23308	
	Rollback:	5145	
Step 2	Import:	22480	
	Commit:	3363	
Step 3	Query 1:	0	4
	Query 2:	1735	264
	Query 3:	0	6
	Query 4:	0	34
	Query 5:	0	719
	Query 6:	224	1048532
	Query 7:	0	67
	Query 8:	8	7790
	Query 9:	1150	27247
	Query 10:	0	4
	Query 11:	0	224
	Query 12:	0	15
	Query 13:	0	472
	Query 14:	112	795970
	Commit:	42	
Step 4	Delete:	811	
	Rollback:	247	
Step 5	Delete:	3509	

	Action	Time (ms)	# of Answers
	Commit:	358	
Step 6	Query 1:	1	0
	Query 2:	139	0
	Query 3:	0	0
	Query 4:	0	0
	Query 5:	0	0
	Query 6:	198	923871
	Query 7:	0	0
	Query 8:	13	4040
	Query 9:	846	22522
	Query 10:	0	0
	Query 11:	0	186
	Query 12:	0	9
	Query 13:	2	413
	Query 14:	103	701364
	Commit:	25	

Table 1: Benchmarking Results

b

The results show that my database implementation is optimised more for query-heavy workloads rather than write-heavy workloads. This makes sense since editing requires acquiring a large number of locks, filling in sets such as the `held_locks` set each transaction, as well as adding tuples to the various group indices. On the other hand, queries typically need to acquire less locks since we can acquire predicate locks that cover multiple tuples, and all these locks are SLocks which can always be acquired.

It is possible to trade off between these by increasing or decreasing the number of indices. For example, we could have made an extra index over the whole relation, or we could have chosen to only create group locks but not group indices. However, I believe my selection to be fairly sensible, avoiding any obviously bad types of queries.

In terms of scalability with respect to the number of open transactions, performance is good when contention is low. If transactions mostly touch different groups/tuples, they can acquire disjoint locks and run largely independently, so increasing the number of open transactions does not hurt throughput much.

The main scalability bottleneck appears once contention rises. More open transactions means larger lock-holder sets and more required-lock edges, and each suspension triggers a DFS deadlock check over that waits-for graph (cost proportional to $V + E$). In addition, coarse predicates such as $R(x, y)$ take relation-level shared locks, which can block many edits at once. So the system scales well in low-conflict workloads, but degrades under high-conflict workloads because more time is spent suspended or in deadlock handling rather than doing useful work; this is the expected trade-off for serializable isolation.